

Sitio: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab>

Exportado: 31 de mayo de 2026

Ziryab

Origen: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab/overview>

ID página: 2162955

Última versión: 8

Descripción

Ziryab es una solución innovadora pensada para agilizar las tareas educativas y administrativas dentro del ámbito de la educación. Incluye todo lo necesario tanto para gestiones administrativas (registro de faltas, creación de horarios...), como para gestionar una clase (subida de contenidos, entrega de tareas...).

Integrantes del grupo

Las fotos del equipo están en Confluence como adjuntos; no se exportaron por API. Ver la página online o `./assets/team/` en el repo TFG-Ziryab.

- Francisco de Asís Cobo Sánchez
- Antonio Salces Alcaraz
- Ángela Mora Mata

Índice (enlaces Confluence)

- [Project Charter](#)

- [Inicio de servidores Angular y Node](#)
 - [Resumen Gestión de proyecto](#)
-

Project Charter

Origen: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab/pages/2261253>

ID página: 2261253

Última versión: 6

Nuestra idea

La aplicación **Ziryab** será el futuro para la **gestión en centros educativos, sencilla, de fácil manejo, amigable al usuario y completa**. Esta aplicación permitirá a los **profesionales de la educación** —ya sean **profesores, orientadores, personal auxiliar y/o administrativo**, así como a los **estudiantes** y sus **padres o tutores**— una gestión integral desde un mismo entorno.

Por el lado de los profesores

- **Organizar las clases**, mandar y recibir tareas.
- **Calificar tareas y exámenes**.
- **Controlar la asistencia** de los estudiantes.
- **Comunicarse con padres o tutores**.
- **Comunicarse internamente** entre profesores, ya sea de **forma libre**, por **departamentos** o con la **administración del centro**.

A los profesores que también ocupen **cargos administrativos** como **secretaría, jefatura de estudios o dirección**, se les dará la opción de **gestionar todo su trabajo desde la misma aplicación**, al estar identificados no solo como docentes sino también por su **puesto específico**.

En el caso de personal con **únicamente tareas administrativas**, tendrán acceso directo a sus funciones desde la aplicación.

Además, Ziryab tendrá en cuenta al **personal de educación especial**, tanto de **centros especializados** como de **alumnos integrados en centros generales**, ofreciendo **facilidades para la comunicación con padres y alumnos** y **adaptándose a diferentes tipos de discapacidad**.

Por el lado de los alumnos

- **Acceder a todas las asignaturas o módulos** cursados.
- Consultar **temarios, tareas y calificaciones**.
- **Conocer** en todo momento el **estado del curso**.
- **Gestionar la firma y recepción de documentos**.
- **Controlar y justificar asistencias y ausencias**.

Parte administrativa

- **Matriculación.**
- **Gestión de becas y solicitudes.**
- **Comunicación con profesores o departamento de orientación.**
- **Entrega y recepción de documentación.**

Todo esto estará acompañado por una **interfaz adaptada a cualquier situación de dificultad por parte de los usuarios.**

En casos especiales, Ziryab permitirá la **interacción directa entre alumnado, profesorado y tutores** siempre que sea necesario.

Project Charter (imagen)

Imagen del charter en Confluence — exportar manualmente desde la página online si hace falta en el PDF.

Diagrama base de datos

Diagrama ER en Confluence — ver también `node/prisma/esquema-relacional.md` en el repositorio backend.

Prototipos Figma

- [Prototipo a PC](#)
- [Prototipo a móvil](#)

Objetivos SMART

Comunicación Alumno Profesor

Implementar un sistema de retroalimentación semanal entre alumno y profesor, mediante un formulario digital, para mejorar la claridad en la entrega de instrucciones y resolución de dudas, alcanzando al menos un 80% de participación de los estudiantes durante los próximos tres meses.

- **Qué/Para quién (S):** Implementar un sistema de retroalimentación semanal mediante formulario digital entre alumno y profesor para todos los estudiantes.
- **Medida de éxito (M):** alcanzar $\geq 80\%$ de participación estudiantil cada semana durante tres meses; feedback recogido en > 10 de 13 semanas posibles.
- **Viabilidad (A):** usando Google Forms y notificaciones por email; esfuerzo estimado 6 h diseño/implantación + 1 h/sem mantenimiento.
- **Relevancia (R):** mejora la claridad de instrucciones y dudas resueltas a tiempo, aumentando satisfacción y rendimiento académico.

- **Plazo (T):** durante los próximos 3 meses (octubre–diciembre); primera recogida antes de finalizar la semana 1.
- **Entregables:** formulario digital, resultados semanales (dashboards), resumen mensual para profesorado, evidencia de envíos/participación.
- **Riesgos y mitigación:** baja participación → recordatorios automáticos y feedback anónimo; problemas técnicos → soporte rápido con plantillas alternativas.

Gestión del curso escolar por parte del profesor

Organizar y publicar en la plataforma educativa el plan de trabajo trimestral con fechas de entrega y criterios de evaluación, asegurando que esté disponible y actualizado el primer día de cada trimestre durante todo el curso.

- **Qué/Para quién (S):** Organizar y publicar en la plataforma educativa (para todos los alumnos y padres) el plan de trabajo trimestral, incluyendo fechas de entrega y criterios de evaluación.
- **Medida de éxito (M):** el plan debe estar online y accesible el primer día de cada trimestre, actualizado al 100% de los trimestres; notificaciones emitidas a todos los destinatarios.
- **Viabilidad (A):** usando la función "Materiales" de la plataforma educativa, esfuerzo estimado 4 h por trimestre para preparación y subida.

- **Relevancia (R):** asegura la transparencia y previsión para estudiantes y familias.
- **Plazo (T):** publicación antes de las 8:00 h del primer día lectivo de cada trimestre.
- **Entregables:** documento o sección del plan disponible en la plataforma, registro de acceso/descarga por al menos el 90% de las familias/estudiantes.

Gestión administrativa para el alumnado

Digitalizar y centralizar el acceso a documentos administrativos (certificados, solicitudes y horarios) en un portal único para los estudiantes, alcanzando un 90% de uso regular por parte del alumnado en un plazo de seis meses.

- **Qué/Para quién (S):** Portal único para todos los estudiantes.
- **Medida de éxito (M):** $\geq 90\%$ de uso regular en seis meses.
- **Viabilidad (A):** portal web con credenciales institucionales; desarrollo estimado 24 h.
- **Relevancia (R):** agiliza trámites y reduce papeleo.
- **Plazo (T):** 6 meses tras el lanzamiento.

- **Entregables:** portal en funcionamiento, registros de acceso, manual de usuario, encuesta de satisfacción.
-

Inicio de servidores Angular y Node

Origen: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab/pages/9404417>

ID página: 9404417

Objetivo

Documentar los pasos necesarios para configurar el entorno de desarrollo e iniciar los servidores de backend (Node.js 19) y frontend (Angular 19), incluyendo configuración de variables de entorno, instalación de dependencias y herramientas relacionadas con Firebase.

Prerrequisitos

- Node.js versión 19 instalada.
- Angular CLI compatible con Angular 19 (recomendada instalación global).
- Acceso al repositorio de los proyectos (Node y Angular).
- Credenciales necesarias para Firebase y otros servicios.

Configuración de variables de entorno (.env)

Backend (Node.js 19)

1. Crear un archivo `.env` en la raíz del proyecto backend si no existe.
2. Definir variables como:
3. `PORT`
4. `NODE_ENV`
5. `DB_URL`
6. Variables relacionadas con Firebase si aplica.
7. Verificar que se use `dotenv` para cargar las variables al arrancar la app.

Frontend (Angular 19)

1. Configurar los archivos de entorno (`environment.ts`, `environment.prod.ts`) o mecanismo `.env` usado por el proyecto, con:
2. URL de la API backend.

3. Configuración Firebase (apiKey, authDomain, etc.).
4. Evitar incluir datos sensibles directamente en el repositorio.

Instalación de dependencias

Instalación global

- Angular CLI compatible con Angular 19:
- `npm install -g @angular/cli@19`
- Firebase CLI para herramientas y despliegues:
- `npm install -g firebase-tools`

Backend (Node 19)

1. En la carpeta backend, ejecutar `npm install` para dependencias del proyecto.
2. Instalar Firebase SDK si se usa:
3. `npm install firebase`

Frontend (Angular 19)

1. En la carpeta frontend, ejecutar `npm install`.
2. Instalar Firebase SDK si se usa:
3. `npm install firebase`

Inicio de los servidores

Backend Node.js 19

1. Confirmar configuración correcta del `.env`.
2. Ejecutar:
3. `npm run dev` (modo desarrollo) o
4. `npm start` (producción local).
5. Verificar funcionamiento en `http://localhost:<PORT>`.

Frontend Angular 19

1. Confirmar correcto apuntado a backend vía variables de entorno.
2. Ejecutar:
3. `ng serve` o
4. `npm run start`.

5. Acceder a <http://localhost:4200> para probar la app Angular.

Resumen Gestión de proyecto

Origen: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab/pages/13500417>

ID página: 13500417

Enlace al tablero Jira del equipo:

<https://g-team-ddm5j4dr.atlassian.net/jira/software/projects/EQ/summary>

Proyecto Jira: EQ en el sitio `g-team-ddm5j4dr.atlassian.net` (distinto del proyecto CURS0 en `franciscocobsan.atlassian.net` usado en desarrollo con Cursor).

Documentación Técnica Frontend (Angular)

Origen: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab/pages/35028993>

ID página: 35028993

Bienvenido/a a la documentación oficial del frontend de la aplicación. Este espacio sirve como punto de entrada para desarrolladores, arquitectos y cualquier miembro del equipo que necesite comprender la estructura, los servicios y la arquitectura visual del proyecto.

Objetivo de la Documentación

El repositorio frontend ha sido rigurosamente documentado utilizando **Compodoc**. Nuestro objetivo es mantener un estándar de calidad estricto para que el mantenimiento, la escalabilidad y la integración de nuevos desarrolladores («onboarding») sea lo más ágil posible.

Hito alcanzado: Actualmente el código cuenta con un **100% de cobertura de documentación**. Todos los módulos, componentes, servicios, guards, interfaces y directivas tienen detallado su propósito, inicialización y retorno.

Acceso a la Documentación (GitHub Pages)

Toda la documentación interactiva, diagramas de módulos, árbol de dependencias y el buscador de componentes se genera de manera automatizada y está alojada en GitHub Pages.

URL actual del proyecto Ziryab:

<https://antoniosalces.github.io/ziryab-compodoc/index.html>

(La página Confluence enlazaba anteriormente a antoniosalces.github.io/documentation/.)

Comandos locales

```
cd angular npm run docs:build # genera estáticos en docs/ npm
run docs:serve # servidor local para revisión
```

Documentación de Sistema de Gestión de Asistencia

Origen: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab/pages/38830081>
ID página: 38830081

1. Decisiones Técnicas y Arquitectura

El módulo de asistencias está diseñado bajo una arquitectura cliente-servidor estricta, priorizando la seguridad y la separación de roles (Profesor vs. Alumno).

Seguridad Core: Toda la comunicación está protegida mediante tokens JWT almacenados en cookies invisibles (HttpOnly). Se implementan validaciones ABAC en el backend (checkSessionOwnership, checkAssistanceOwnership) para asegurar que un profesor solo modifica sus propias clases, y un alumno solo ve sus propias faltas.

2. Esquema de Base de Datos

Campo	Tipo	Nulo	Descripción	Relación
id	Int	No	Identificador único de la asistencia	-
status	Enum	No	PRESENT, ABSENT, LATE, EXCUSED	-
justificacionUri	String	Si	Ruta local del servidor	-
idSession	Int	No	ID de la sesión de clase impartida	SessionClass

idStrudentEnrollment	Int	No	ID de la matrícula del alumno	StudentEnrollm
----------------------	-----	----	-------------------------------	----------------

3. Backend

- **POST /api/assistances/bulk:** Permite registrar la asistencia de toda una clase en ráfaga. Valida estados y propiedad de la sesión.
- **GET /api/assistances/session/:idSession:** Devuelve la lista de alumnos y sus estados para una clase específica.
- **PATCH /api/assistances/assistancestatus/:id:** Actualiza el estado de una falta puntual

PATCH /api/assistances/justify/:id: El profesor aprueba el justificante y cambia el estado a **EXCUSED**

GET /api/assistances/my-absences: Devuelve los registros donde el alumno tiene **ABSENT**, **LATE** o **EXCUSED** (ignora los **PRESENT**).

- **POST /api/assistances/:id/justification-document:** Recibe un archivo mediante **multipart/form-data** (PDF/JPG/PNG), lo guarda en disco y actualiza el **justificationUri**.
- **GET /api/assistances/:id/justification-status:** Consulta el estado de la validación de su falta.

4. Frontend

Componentes del Profesor

- **ClasesProfesorComponent**: Pinta tarjetas con las asignaturas del profesor autenticado. Redirige al componente Temario pasando el `subjectId` por Query Params.

Componentes del Alumno

- **FichaUsuarioComponent**: Dashboard del estudiante. Muestra el historial mediante pills de colores (Rojo = Falta, Naranja = Retraso, Verde = Justificada).
- **JustificarFaltaModalComponent**: Componente de presentación.
- **Lógica**: Valida del lado del cliente el tamaño del archivo (`< 1MB`) y la extensión permitida.
- **Comunicación**: Usa `@Input` para recibir los datos y `@Output` para notificar éxito a la vista padre mostrando animaciones de carga.

EQ-154 · Guía de despliegue actual en Render

Origen: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab/pages/43745281>

ID página: 43745281

Documento operativo para desplegar **backend (Node/Express)** y **frontend (Angular)** en Render, con checklist de validación, rollback y protocolo manual seguro para credenciales/variables.

1) Alcance

- Backend API: repositorio `tfg/node`
- Frontend web: repositorio `tfg/angular`
- Entorno objetivo: Render (servicios independientes)

2) Inventario técnico actual

Backend (node)

- Runtime: Node.js + TypeScript + Express

- Build command: `npm run build`
- Start command: `npm start`
- Puerto de escucha: `PORT` (por defecto 3000)
- Healthcheck: `GET /health`
- Docs API: `GET /api-docs`
- Config de CORS: usa `FRONTEND_URL`
- Seguridad CSP (helmet): `connectSrc` actualmente restringido a `'self'` (se debe ampliar con dominio real de Render si aplica en navegadores/clientes remotos)

Frontend (angular)

- Build command: `npm run build`

- Artefacto: `dist/login-en-angular`
- URL API en desarrollo: `http://localhost:3000/api` (config local)
- Recomendación: en despliegue usar variable/archivo de entorno de producción apuntando a la URL pública del backend en Render.

3) Variables de entorno requeridas (backend)

Obligatorias en Render (sin valores en esta página):

- `NODE_ENV` (producción)
- `PORT`
- `DATABASE_URL`
- `JWT_SECRET`
- `JWT_EXPIRY`

- `FIREBASE_PROJECT_ID`
- `FIREBASE_PRIVATE_KEY`
- `FIREBASE_CLIENT_EMAIL`
- `FRONTEND_URL`

Notas críticas

- `JWT_SECRET` debe tener al menos 32 caracteres.
- `DATABASE_URL` debe ser URL válida.
- `FIREBASE_PRIVATE_KEY` requiere respetar formato de saltos de línea al cargarla en Render.
- `FRONTEND_URL` debe coincidir con el dominio público real del frontend en Render para evitar bloqueo CORS.

4) Procedimiento de despliegue en Render

Backend

1. Conectar repositorio `tfg/node` al servicio web en Render.
2. Configurar:
3. Build: `npm run build`
4. Start: `npm start`
5. Cargar variables de entorno obligatorias (solo nombres en este documento).
6. Desplegar rama objetivo (`main` o la acordada).
7. Verificar:
8. `GET /health` responde `{ ok: true }`

9. GET /api-docs carga correctamente
10. Endpoints críticos responden 2xx/4xx esperados

Frontend

1. Conectar repositorio tfg/angular al servicio estático/web en Render.
2. Configurar build: npm run build.
3. Publicar carpeta de salida (dist/login-en-angular) según configuración del servicio.
4. Configurar URL de API en entorno de producción para apuntar al backend Render.
5. Verificar login y navegación de pantallas clave.

5) Checklist de validación post-deploy

- Backend responde en `/health`.
- Swagger en `/api-docs` accesible.
- CORS correcto entre frontend y backend desplegados.
- Flujo de autenticación funcional (login + consumo de endpoint protegido).
- Logs de Render sin errores críticos de arranque.
- Variables obligatorias cargadas en backend.

6) Rollback básico

1. Identificar último deploy estable en Render.
2. Revertir a la versión estable (rollback/redeploy del commit previo).
3. Revalidar `/health`, login y endpoint protegido.

4. Si el fallo es de configuración, restaurar variables previas y redeploy.

7) Troubleshooting rápido

- **Error 500 al iniciar backend:** revisar `DATABASE_URL`, `JWT_SECRET`, variables Firebase.
- **CORS bloqueado en frontend:** validar `FRONTEND_URL` en backend.
- **Autenticación Firebase falla:** revisar `FIREBASE_PRIVATE_KEY` y `FIREBASE_CLIENT_EMAIL`.
- **Frontend no consume API:** comprobar URL base de API en configuración de producción.
- **CSP/connect-src bloquea conexiones:** actualizar política de Helmet para incluir dominio Render necesario.

8) Protocolo manual seguro para credenciales y variables

Qué se comparte

- Solo: nombres de variables, responsables, entorno destino (dev/stage/prod) y fecha de actualización.

Qué NO se comparte

- Nunca publicar valores reales en Jira, Confluence, chats de grupo, commits, PRs o capturas.
- Nunca guardar secretos en archivos versionados (.env, JSON de credenciales, etc.).

Canal permitido

- Canal privado 1:1 o reunión breve con persona autorizada.

- Confirmación explícita de recepción y carga correcta en Render.

Proceso recomendado (manual seguro)

1. Responsable A prepara rotación/actualización de secreto.
2. Responsable B recibe por canal privado permitido.
3. Responsable B carga secreto en Render y confirma sin reenviar el valor.
4. Ambos validan servicio (`/health` + login).
5. Registrar en Confluence/Jira solo metadatos: variable, fecha, responsable, estado (sin valor).

9) Trazabilidad

- Ticket Jira: [EQ-154](#)

- URLs producción actuales: ziryabback.onrender.com · ziryabfront.onrender.com
-

EQ-197 · Creación automática de StudentTask al publicar tarea

Origen: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab/pages/43712514>

ID página: 43712514

Objetivo

Alinear la lógica de backend para que los `StudentTask` se creen **cuando la tarea se publica**, no siempre al crearla.

Ticket Jira

- EQ-197
- Descripción: al ejecutar `POST /tasks`, generar un `StudentTask PENDING` por cada alumno inscrito en el grupo asociado a la asignación.

Cambios implementados

1) `task.service.ts`

`create()`

- Se añade soporte para `isPublished` en `CreateTaskData`.

- Se persiste `isPublished` en la entidad `Task` (default `false` si no se envía).
- La creación de `StudentTask` se ejecuta **solo si** `task.isPublished === true`.

`update()`

- Se añade soporte para `isPublished` en `UpdateTaskData`.
- Si la tarea pasa de `isPublished=false` a `isPublished=true`, se crean automáticamente `StudentTask PENDING` para todos los alumnos `ENROLLED` de la asignatura/grupo/año.
- Se usa `skipDuplicates: true` para evitar errores por la constraint única (`idTask, idStudentEnrollment`).

2) `task.controller.ts`

- Se añade parseo/validación de `isPublished (true/false)` para `POST /api/tasks` y `PATCH /api/tasks/:id`.

- Si `isPublished` es inválido, devuelve `400` con mensaje claro.

3) Tests

- Se actualiza `task.service.spec.ts`:
- Caso: crea tarea publicada y genera `StudentTask`.
- Caso nuevo: crea tarea no publicada y **no** genera `StudentTask`.
- Caso nuevo: al publicar desde `update`, genera `StudentTask` con `skipDuplicates: true`.

Resultado funcional

- Tarea no publicada: no se generan entregas.
- Tarea publicada en creación: se generan entregas `PENDING`.

- Tarea publicada posteriormente (PATCH): se generan entregas PENDING para matriculados faltantes.

Verificación ejecutada

- Comando: `npm test -- src/modules/task/task.service.spec.ts`
 - Resultado: 10/10 tests OK.
-

Decisión: Vulnerabilidad de firebase-admin (Issue #5)

Origen: <https://g-team-ddm5j4dr.atlassian.net/wiki/spaces/Ziryab/pages/40730625>

ID página: 40730625

Contexto

Al evaluar la seguridad de las dependencias (`npm audit`) en el proyecto backend alojado en Node.js, se detecta un aviso relacionado con el paquete `firebase-admin` (versión actual en uso: `13.6.0`).

Análisis de la Vulnerabilidad

- **Severidad:** Baja (LOW). CVSS Score: 3.3.
- **Origen Real:** La vulnerabilidad tiene origen en el paquete `@tootallnate/once` (menor a la versión `3.0.1`), que presenta un problema denominado "Incorrect Control Flow Scoping" (ver [GHSA-vpq2-c234-7xj6](#)).
- **Cadena de dependencias:** La vulnerabilidad llega a `firebase-admin` como dependencia transitiva a través de la siguiente cadena:
`@tootallnate/once` → `http-proxy-agent` → `teeny-request` → `retry-request` → `google-gax` → `@google-cloud/storage` y `@google-cloud/firestore` → `firebase-admin`.
- **Posible solución de NPM:** Al ejecutar una reparación automática (es decir, permitiendo "breaking changes"), CLI de npm propone degradar (*downgrading*) `firebase-admin` a la versión `10.3.0`.

Evaluación y Decisión

1. **Impacto:** La falla califica como "Low severity", relacionada con funciones dependientes de proxies y bajo escenarios exóticos y muy específicos. Afecta mínimamente a los procesos usuales de backend.
2. **Riesgo del fix:** Bajar el paquete principal `firebase-admin` de la versión `13.6.0` a la `10.3.0` (un retroceso de tres versiones mayores) supone perder un gran número de importantes parches de seguridad, optimizaciones de rendimiento y soporte de nuevas y críticas APIs de Firebase. El impacto sería destructivo y mucho más riesgoso para la estabilidad y seguridad integral de la aplicación.
3. **Decisión Adoptada: Asumir y aceptar el riesgo.** Hemos decidido **mantener** `firebase-admin` en la versión actual (`13.6.0`) y obviar la recomendación de arreglo rompedor.
4. **Siguientes pasos:** Monitorizaremos futuras actualizaciones de `firebase-admin` que incorporen parches a nivel de sus dependencias transicionales (sobre Google Cloud SDKs y `@tootallnate/once`), momento en el que actualizaremos con normalidad sin requerir downgrade.